14. Random en Unity

Unity incluye la clase Random dentro del espacio de nombres UnityEngine, y permite generar números aleatorios, vectores aleatorios, rotaciones aleatorias e incluso elegir elementos aleatorios de una lista.
El uso de valores aleatorios es fundamental para introducir variedad en el juego: aparición de enemigos, dispersión de disparos, spawn de objetos, movimientos impredecibles, etc.

14.1. Random.Range

Es el método más utilizado y permite obtener un número aleatorio entre dos valores.

Enteros

```
int numero = Random.Range(0, 5);
```

Devuelve: **0, 1, 2, 3 o 4**
(El valor máximo no está incluido)

Floats

```
float numero = Random.Range(0f, 5f);
```

Devuelve un float entre **0 y 5**, incluyendo ambos extremos.
(En floats, el valor máximo sí está incluido)

14.2. Uso de Random en los scripts del proyecto

14.2.1. GameManager: aparición de rocas aleatoriamente

```
yield return new WaitForSeconds(Random.Range(2f,4f));
```

Aquí, el tiempo entre spawns de rocas es aleatorio.

Posición y rotación aleatoria de la roca

```
Vector3 posicion = new Vector3(Random.Range(-9f, 9f),
Random.Range(-5f, 5f), 0);

Instantiate(prefabRoca, posicion, Quaternion.Euler(0, 0,
Random.Range(0, 360)));
```

Se generan:

- Posición X aleatoria
- Posición Y aleatoria
- Rotación aleatoria (0–360°)

Esto consigue un sistema de aparición impredecible y más natural.

14.3. Otros usos comunes de Random

Random.insideUnitCircle

Devuelve un punto aleatorio dentro de un círculo de radio 1:

```
Vector2 punto = Random.insideUnitCircle;
```

Ideal para dispersión o efectos.

Random.insideUnitSphere

Lo mismo, pero en 3D.

Random.onUnitSphere

Devuelve un punto aleatorio en la superficie de una esfera.

Random.value

Devuelve un float entre 0 y 1.
Muy útil para "probabilidades":

```
if (Random.value < 0.1f)
{
    // 10% de probabilidad
}
```

14.4. Semillas aleatorias (Random.InitState)

Unity permite usar una semilla fija para reproducir resultados aleatorios:

Random.InitState(10);

Esto hace que todas las llamadas a Random produzcan siempre la misma secuencia de valores.
Es útil para debugging o para juegos que necesitan mundos reproducibles.

14.5. Random y rotaciones

Para generar ángulos aleatorios:

```
float angulo = Random.Range(0f, 360f);
Quaternion rot = Quaternion.Euler(0, 0, angulo);
```

Para un dispersor de balas, partículas o enemigos voladores, esto es fundamental.